

Official website: <https://redis.io/>

What is Redis?

Redis is a **storage system that keeps data in your computer's memory (RAM)** instead of on a hard disk. Think of it like keeping frequently used items on your desk rather than in a filing cabinet across the room. Its main purpose is to help applications access data extremely fast.

The name Redis stands for "**RE**mote **D**ictionary **S**erver" - it's basically a smart storage box that you can access over a network.

We can implement redis in all modern popular languages like Java, python, Go, javascript, typescript etc.

Real-Life Example

Imagine you're building a social media app like Twitter. When someone visits a popular user's profile:

Without Redis:

- Your app asks the database: "Get me this user's profile information"
- The database searches through millions of records on the hard disk
- This takes 100 milliseconds (which feels slow when millions of people are using your app)
- If 1,000 people view the same profile in one minute, you make 1,000 slow database trips

With Redis:

- First time: Your app gets the data from the database and saves a copy in Redis
- Next 999 times: Your app gets the data from Redis in just 1 millisecond
- The data sits in memory, ready to be grabbed instantly
- Your app feels much faster, and your database isn't overwhelmed

Historical Context

1. What were the limitations before Redis?

Before Redis (and similar tools), developers faced these problems:

- **Slow databases:** Traditional databases stored everything on hard disks. Reading from a disk is like walking to a library - it takes time.
- **Repeated slow queries:** Apps would ask the database the same questions over and over, like asking "What's 2+2?" a thousand times instead of remembering the answer.

- **Overloaded databases:** When many users accessed an app at once, the database would struggle, like a single cashier serving hundreds of customers.
- **Limited options for temporary data:** Storing temporary information (like "user is logged in") required complex workarounds.

2. What specific problems does Redis solve?

Redis solves these key problems:

- **Speed:** Gets data in microseconds instead of milliseconds because memory is 100+ times faster than disk storage
- **Reduces database load:** Acts like a helper that remembers recent answers, so the main database can rest
- **Stores temporary data easily:** Perfect for things that don't need to last forever, like shopping carts or login sessions
- **Simple to use:** Stores data as key-value pairs (like a dictionary: "user123" → user's information)
- **Handles expiration:** Can automatically delete old data after a set time (like notes that self-destruct)

3. How has Redis changed practices?

Since Redis was created in 2009, it has changed how developers build applications:

- **Caching became standard:** Almost every major website now uses Redis or similar tools to cache (temporarily store) frequently accessed data
- **Real-time features became easier:** Chat applications, live notifications, and leaderboards became simpler to build because Redis handles rapid updates well
- **Better user experience:** Websites load faster because data is retrieved from memory instead of disk
- **Cost savings:** Companies need fewer database servers because Redis handles much of the load
- **New patterns emerged:** Developers created new ways to build apps, like using Redis for rate limiting (controlling how often someone can do something) or as a message queue (managing tasks to be done)

4. How to use redis in go lang?

1) Quick prerequisites

Go (1.18+) installed, run redis server(docker is best option `docker run -d --name redis -p 6379:6379 redis:latest`)

2) Add the Go Redis client

There are multiple redis client options in Go like Go-redis, Redigo, Radix etc. Go-redis is the most popular. We will go like this.

```
go get github.com/redis/go-redis/v9
```

3) Create main.go

```

package main

import (
    "context"
    "fmt"
    "time"

    "github.com/redis/go-redis/v9"
)

var ctx = context.Background()

func main() {
    rdb := redis.NewClient(&redis.Options{
        Addr:      "localhost:6379",
        Password: "", // set if you enabled AUTH
        DB:        0,
    })
    defer rdb.Close()

    // SET (no expiry)
    if err := rdb.Set(ctx, "hello", "world", 0).Err(); err != nil {
        panic(err)
    }

    // GET
    val, err := rdb.Get(ctx, "hello").Result()
    if err == redis.Nil {
        fmt.Println("key does not exist")
    } else if err != nil {
        panic(err)
    } else {
        fmt.Println("hello =", val)
    }

    // Example with expiration
    _ = rdb.Set(ctx, "session:123", "userdata", 30*time.Minute)
}

```

4) Best practices: context & timeouts

```

ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
defer cancel()
err := rdb.Get(ctx, "somekey").Err()

```

5) Storing Go structs (JSON)

```
type User struct {
    ID    int    `json:"id"`
    Name string `json:"name"`
}
b, _ := json.Marshal(user)
rdb.Set(ctx, "user:42", b, 10*time.Minute)

// read back
bs, _ := rdb.Get(ctx, "user:42").Bytes()
var u User
json.Unmarshal(bs, &u)
```

6) Common data structures

```
// Hash
rdb.HSet(ctx, "user:42:profile", "name", "Rohit", "age", 30)
vals, _ := rdb.HGetAll(ctx, "user:42:profile").Result()

// List as queue
rdb.LPush(ctx, "jobQ", "job1")
job, _ := rdb.BRPop(ctx, 0, "jobQ").Result() // blocking pop
```

7) Pub / Sub (example)

Subscriber

```
pubsub := rdb.Subscribe(ctx, "notifications")
defer pubsub.Close()

ch := pubsub.Channel()
for msg := range ch {
    fmt.Println("channel:", msg.Channel, "payload:", msg.Payload)
}
```

Publisher

```
rdb.Publish(ctx, "notifications", "hello subscribers")
```

There are few important topics in redis Transactions & optimistic locking, Pipelines, Redis Cluster & Sentinel, Connection pool & tuning, Running Redis locally (dev) & testing, Production considerations, repo skeleton.

4. How to use redis in java?

1) Quick prerequisites

- Java 11+
- Maven or Gradle for dependency management.
- run redis server(docker is best option)

```
docker run -d --name redis -p 6379:6379 redis
```

2) Add the JAVA Redis client

There are multiple options: Lettuce, Jedis, Redisson, Spring Data Redis. We'll go with Jedis.

3) Redis example

```
import redis.clients.jedis.Jedis;
import redis.clients.jedis.JedisPool;
import redis.clients.jedis.JedisPoolConfig;

public class JedisExample {
    public static void main(String[] args) {
        JedisPool pool = new JedisPool(new JedisPoolConfig(), "localhost", 6379);
        try (Jedis jedis = pool.getResource()) {
            jedis.set("hello", "world");
            String val = jedis.get("hello");
            System.out.println("hello = " + val);
        } finally {
            pool.close();
        }
    }
}
```

These are very popular and common redis options = SET / GET, Hashes, Lists, Sets / Sorted Sets, Pub/Sub, Streams, Transactions, Pipelining, Lua scripting. These are supported by every language.

Redis competitors (Exclude redis stream competitors)

Memcached, Hazelcast, Ehcache, Apache Ignite, Aerospike, Couchbase etc. but Redis is the most popular.

What is Redis Stream?

Think of Redis Stream like a **never-ending notebook** where you write messages one after another. Each message gets a unique page number, and anyone can read from any page they want.

Real-World Example

Imagine a restaurant kitchen:

- **Old way:** Waiter shouts "One pizza!" and chef makes it. If chef is busy, order is lost.
- **Redis Stream way:** Waiter writes order on a ticket with timestamp. Chef reads tickets one by one. If chef is sick, another chef can continue from where they stopped.

Simple Explanation with Example

Stream: "orders"

```
Entry 1: {customer: "John", item: "Pizza", time: "10:00 AM"}
Entry 2: {customer: "Mary", item: "Burger", time: "10:05 AM"}
Entry 3: {customer: "Sam", item: "Pasta", time: "10:10 AM"}
```

Limitations STILL in Redis Streams

1. Memory Problem 🧠

- Stores everything in RAM (computer memory)
- If you store millions of messages → needs lots of memory → expensive!
- Example: Storing 1 million messages might need several GBs of RAM

2. No Complex Queries 🔍

- Can't search like: "Find all pizza orders by John in March"
- Can only read in **time order** (oldest to newest)
- **Example:** To find specific message, you must read all messages and filter yourself

3. Time-Based Deletion Only ⌚

- Can't delete specific message easily
- Can only trim from beginning: "delete all before ID X"

When to Use Redis Streams?

✅ Good for:

- Activity feeds (like Twitter timeline)
- Sensor data (temperature readings)
- Log processing
- Event notifications
- Chat messages
- Real-time analytics

✗ Not good for:

- Long-term data storage (use database)
- Complex searching (use Elasticsearch)
- Very large files (use S3/file storage)
- When you need 100% guarantee no data loss (use Kafka)

Redis Streams competitors

Kafka (most used), RabbitMQ, NATS, Apache Pulsar, AWS Kinesis, Google Pub/Sub, Azure Event Hubs.